

Pràctica 2. Supermercat

UD 2. Programació multifil

Programació de Serveis
i Processos

CFGS Desenvolupament
d'Aplicacions Multiplataforma



IES Jaume II El Just
Tavernes de la Valldigna
Departament d'Informàtica

Continguts

Pràctica 2: Sistema de preparació de comandes d'un supermercat	2
1. Context	2
2. Objectius	2
3. Model del problema	3
4. El productor	3
Productors configurats	4
5. El consumidor	5
6. La zona de preparació	5
7. Quan la zona està plena	6
8. Quan la zona està buida	6
9. Ordre de les comandes	7
10. Finalització dels productors	8
11. El programa principal	8
12. Coordinació entre fils	9
synchronized	9
wait()	9
notifyAll()	10
join()	10
13. Resultat esperat	10
17. Què s'ha d'entregar	11

Pràctica 2: Sistema de preparació de comandes d'un supermercat

1. Context

Un supermercat disposa d'una aplicació que permet als seus clients realitzar comandes en línia.

Cada vegada que un client confirma una comanda, aquesta entra en una **zona de preparació** on queda pendent fins que un dels treballadors del supermercat la recull i la prepara.

El funcionament del sistema és concurrent:

- Diversos clients poden realitzar comandes al mateix temps.
- Diversos treballadors poden preparar comandes simultàniament.
- La zona de preparació té una **capacitat limitada**.
- Un client no pot deixar una comanda a la zona si aquesta està plena.
- Un treballador no pot retirar una comanda si no n'hi ha cap disponible.
- Quan una situació canvia, els fils que estan esperant han de poder continuar la seua execució.

En aquesta pràctica, els **clients seran els productors** i els **treballadors seran els consumidors**.

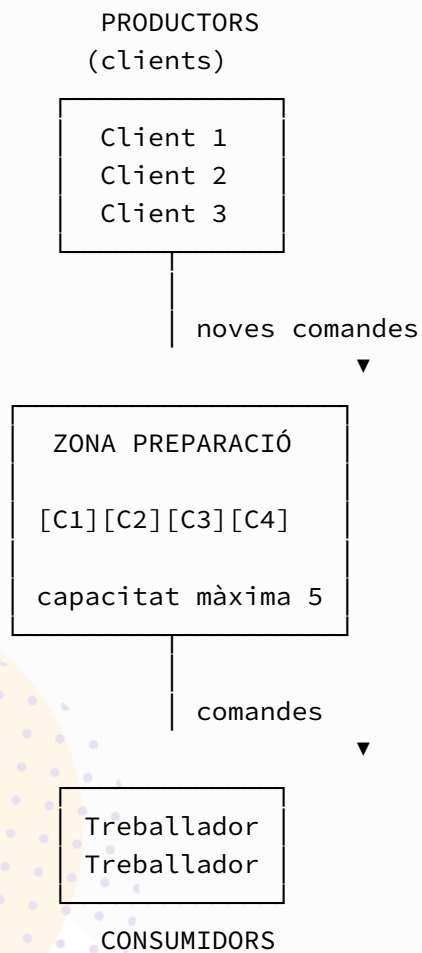
2. Objectius

En finalitzar la pràctica hauràs de ser capaç de:

- Crear i gestionar fils d'execució en Java.
- Utilitzar la interfície `Runnable`.
- Compartir un recurs entre diversos fils.
- Identificar i protegir una secció crítica.
- Utilitzar `synchronized`.
- Utilitzar `wait()` per bloquejar un fil quan no pot continuar.
- Utilitzar `notifyAll()` per avisar els fils que estan esperant.
- Treballar amb un buffer de capacitat limitada.
- Coordinar diversos productors i consumidors.
- Utilitzar `join()` per esperar la finalització dels fils.
- Comprendre què ocorre quan el buffer està ple o buit.
- Garantir que totes les comandes es processen correctament.

3. Model del problema

El sistema està format per tres tipus d'elements:



La ZonaPreparacio és el **recurs compartit** entre tots els fils.

4. El productor

Cada productor representa un client que realitza diverses comandes.

En el projecte proporcionat ja tens preparada la classe `Productor`, però hauràs d'implementar el mètode `run()`.

Cada productor haurà de:

1. Generar el nombre de comandes que se li ha indicat.
2. Crear cada objecte `Comanda`.
3. Assignar-li un identificador.
4. Assignar-li el client corresponent.
5. Assignar-li un nombre aleatori de productes.
6. Mostrar per pantalla la comanda generada.
7. Introduir la comanda en la `ZonaPreparacio`.
8. Esperar un temps aleatori abans de generar la següent comanda.
9. Gestionar correctament les possibles interrupcions.
10. Informar quan haja finalitzat.

Productors configurats

El programa principal haurà de crear tres productors:

Fil	Productor	Nombre de comandes
Client-1	Productor 1	3
Client-2	Productor 2	4
Client-3	Productor 3	5

Per tant, en total es generaran:

$3 + 4 + 5 = 12$ comandes, però en podrien generar les que vulguren, sols és un exemple

5. El consumidor

Cada consumidor representa un treballador del supermercat encarregat de preparar les comandes.

En el projecte proporcionat ja tens preparada la classe `Consumidor`, però hauràs d'implementar el mètode `run()`.

Cada consumidor haurà de:

1. Intentar retirar una comanda de la zona.
2. Si no hi ha comandes disponibles, esperar.
3. Quan obtinga una comanda, comprovar si és `null`.
4. Si és `null`, significa que no queden comandes i que els productors ja han finalitzat.
5. Si obté una comanda, simular-ne la preparació.
6. Mostrar per pantalla el procés de preparació.
7. Esperar un temps aleatori durant la preparació.
8. Mostrar que la comanda ha estat preparada.
9. Continuar processant comandes fins que no en queden.

Hi haurà **2 treballadors**:

- Treballador-1
- Treballador-2

6. La zona de preparació

La classe `ZonaPreparacio` representa el **buffer compartit**.

La seua capacitat màxima serà de:

5 comandes

En el projecte trobaràs els atributs i el constructor ja implementats.

També trobaràs preparades les definicions dels tres mètodes principals:

```
public synchronized void afegirComanda(Comanda comanda)
    throws InterruptedException
```

```
public synchronized Comanda retirarComanda()  
    throws InterruptedException
```

```
public synchronized void finalitzarProductors()
```

La teua tasca serà implementar-los seguint les indicacions dels TODO que trobaràs en el codi.

7. Quan la zona està plena

Imagina que la zona de preparació ja conté cinc comandes:

```
[ C1 ][ C2 ][ C3 ][ C4 ][ C5 ]
```

Un productor intenta afegir una nova comanda.

No pot fer-ho perquè la zona està plena.

Per tant, el productor haurà de quedar **esperant** fins que algun consumidor retire una comanda.

Quan això ocorregui, haurà de poder continuar.

Per implementar aquesta situació hauràs d'utilitzar els mecanismes de coordinació entre fils estudiats en la unitat.

Important: la comprovació de la condició d'espera s'ha de fer amb `while`, no amb `if`.

8. Quan la zona està buida

Ara imagina la situació contrària:

```
[ ]
```

Un treballador intenta retirar una comanda però no n'hi ha cap disponible.

En aquest cas, el treballador haurà de quedar **esperant**.

Quan algun productor introduísca una nova comanda, els fils que estiguen esperant hauran de poder tornar a comprovar la situació.

De nou, hauràs d'utilitzar els mecanismes de coordinació entre fils.

9. Ordre de les comandes

Les comandes s'han de gestionar en ordre **FIFO**.

FIFO significa:

First In, First Out

És a dir, la primera comanda que entra a la zona ha de ser la primera que isca.

Per exemple:

Entrada:

C1 → C2 → C3 → C4

Eixida:

C1 → C2 → C3 → C4

No es pot retirar una comanda situada al mig de la cua mentre hi haja comandes anteriors pendents.

10. Finalització dels productors

Hi ha un problema que cal solucionar.

Els productors generen un nombre finit de comandes.

Per tant, arriba un moment en què **ja no es generaran més comandes**.

Els consumidors, però, no saben automàticament que els productors han acabat.

Per solucionar-ho, la zona disposa de la variable:

```
private boolean productorsFinalitzats;
```

i del mètode:

```
public synchronized void finalitzarProductors()
```

El programa principal haurà d'avisar la zona quan tots els productors hagen finalitzat.

A partir d'aquest moment:

- si encara queden comandes, els consumidors les han de preparar;
- si no queda cap comanda, els consumidors han de finalitzar.

El mètode `retirarComanda()` utilitzarà el valor `null` per indicar aquesta situació.

11. El programa principal

La classe `Main` conté inicialment la creació de la zona:

```
ZonaPreparacio zona = new ZonaPreparacio(5);
```

La resta del programa haurà d'implementar-la.

Els `TODO` del fitxer t'indiquen les tasques que has de realitzar.

En particular, hauràs de:

1. Crear els tres fils productors.
2. Crear els dos fils consumidors.

3. Assignar un nom a cada fil.
 4. Iniciar tots els fils.
 5. Esperar que finalitzen els productors utilitzant `join()`.
 6. Informar la zona que els productors han finalitzat.
 7. Esperar que finalitzen els consumidors.
 8. Mostrar el missatge final del programa.
-

12. Coordinació entre fils

En aquesta pràctica hauràs d'utilitzar els mecanismes de sincronització de Java.

Recorda el significat dels principals mètodes:

synchronized

Permet protegir l'accés al recurs compartit.

wait()

Un fil utilitzarà `wait()` quan **no puga continuar perquè la situació actual no ho permet**.

Exemples:

- Un productor intenta afegir una comanda però la zona està plena.
 - Un consumidor intenta retirar una comanda però la zona està buida i encara poden arribar-ne més.
-

notifyAll()

S'utilitzarà quan una operació haja modificat l'estat de la zona i puga permetre que altres fils continuen.

Per exemple:

- Un productor afegeix una comanda → pot haver-hi consumidors esperant.
- Un consumidor retira una comanda → pot haver-hi productors esperant.
- Els productors finalitzen → pot haver-hi consumidors esperant que ja no rebran més comandes.

join()

El fil principal utilitzarà `join()` per esperar que altres fils finalitzen.

13. Resultat esperat

El resultat exacte dependrà de l'execució concurrent i dels temps aleatoris, per la qual cosa **no has d'esperar sempre el mateix ordre dels missatges**.

Un possible resultat podria ser:

```
[Client-1] Genera Comanda{id=100, client='Client-1', productes=4}
[Client-2] Genera Comanda{id=200, client='Client-2', productes=7}
[Client-3] Genera Comanda{id=300, client='Client-3', productes=2}
```

```
[Treballador-1] Retira comanda 100
[Treballador-1] Preparant comanda 100
```

```
[Treballador-2] Retira comanda 200
[Treballador-2] Preparant comanda 200
```

```
[Client-1] Genera Comanda{id=101, client='Client-1', productes=6}
```

...

[SISTEMA] Ja no arribaran més comandes.

[Treballador-1] Comanda 304 preparada.

[Treballador-2] Comanda 204 preparada.

Sistema finalitzat.

L'ordre concret pot canviar en cada execució.

Això és una conseqüència natural de l'execució concurrent dels fils.

17. Què s'ha d'entregar

Cal lliurar per aules el projecte Java que se vos proporciona, completament implementat i funcional.

El programa haurà de poder executar-se sense errors i haurà de complir totes les condicions indicades en aquest enunciat.